[Huyen Nguyen] Userspace Debugging Toolkit: GDB, strace & Valgrind

Program

```
void task_1_segfault()
{
    int *null_ptr = NULL;
    *null_ptr = 100;
}
int *shared_ptr = NULL;
void* allocator_thread(void* arg)
{
    while (1)
    {
        shared_ptr = (int*)malloc(sizeof(int));
        *shared_ptr = 100;
        free(shared_ptr);
        shared_ptr = NULL;
    }
    return NULL;
}
void* worker_thread(void* arg)
{
    while (1)
    {
        if (shared_ptr != NULL)
        {
            *shared_ptr = 999;
        }
    }
    return NULL;
```

```
}
void task_2_race_condition()
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, allocator_thread, NULL);
    pthread_create(&t2, NULL, worker_thread, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
void* deadlock_A(void* arg)
{
    pthread_mutex_lock(&lock1);
    sleep(1);
    pthread_mutex_lock(&lock2);
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
void* deadlock_B(void* arg)
{
    pthread_mutex_lock(&lock2);
    sleep(1);
    pthread_mutex_lock(&lock1);
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}
void task_3_strace(int hang)
{
    if (hang)
    {
        pthread_t t1, t2;
        pthread_create(&t1, NULL, deadlock_A, NULL);
        pthread_create(&t2, NULL, deadlock_B, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
    }
```

```
    else
    {
        FILE *f = fopen("/root/secret_admin_file.txt", "r");
        if (!f)
        {
            perror("fopen failed");
        }
        pid_t pid = fork();
        if (pid == 0)
        {
            printf("[Child PID: %d]\n", getpid());
            sleep(1);
            exit(0);
        }
        else
        {
            wait(NULL);
        }
    }
}
void task_4_valgrind()
{
    int *leak_arr = (int*)malloc(10 * sizeof(int));
    leak_arr[0] = 42;
    printf("[Leak] First value: %d\n", leak_arr[0]);
    int *uaf_ptr = (int*)malloc(sizeof(int));
    *uaf_ptr = 100;
    free(uaf_ptr);
    printf("[UAF] Value after free: %d\n", *uaf_ptr);
    *uaf_ptr = 999;
}
int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("  Task 1 (Segfault & Core dump):   %s 1\n", argv[0]);
        printf("  Task 2 (Race Condition Crash):  %s 2\n", argv[0]);
        printf("  Task 3 (strace syscalls):       %s 3\n", argv[0]);
        printf("  Task 3 (strace program hangs):  %s 3 --hang\n", argv[0]);
```

```
        printf("  Task 4 (Valgrind Memcheck):     %s 4\n", argv[0]);
        return 0;
    }
    int choice = atoi(argv[1]);
    switch (choice)
    {
        case 1:
            task_1_segfault();
            break;
        case 2:
            task_2_race_condition();
            break;
        case 3:
            if (argc > 2 && strcmp(argv[2], "--hang") == 0)
            {
                task_3_strace(1);
            }
            else
            {
                task_3_strace(0);
            }
            break;
        case 4:
            task_4_valgrind();
            break;
        default:
            break;
    }
    return 0;
}
```

1. GDB

1.1. Lab 1

Debug Live with GDB (Breakpoints + Backtrace)

Set a breakpoint and run

```
(gdb) break task_1_segfault
Breakpoint 1 at 0x1351: file summary.c, line 11.

(gdb) r 1
Starting program: /home/huyen/Downloads/Session 6/summary 1

Breakpoint 1, task_1_segfault () at summary.c:11
11□    int *null_ptr = NULL;
```

Next line

```
(gdb) n
12□    *null_ptr = 100;
(gdb) n

Program received signal SIGSEGV, Segmentation fault.
0x000055555555535d in task_1_segfault () at summary.c:12
12□    *null_ptr = 100;
```

Print the call stack and inspect variables

```
(gdb) bt
#0  0x000055555555535d in task_1_segfault () at summary.c:12
#1  0x00005555555557c3 in main (argc=2, argv=0x7fffffffdd98) at summary.c:121
(gdb) print null_ptr
$1 = (int *) 0x0
```

Core dump

Load the core file

```
Core was generated by `/home/huyen/Downloads/Session 6/summary 1'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x000055555555535d in task_1_segfault () at summary.c:12
12□    *null_ptr = 100;
```

Inspect using the bt command

```
(gdb) bt
#0  0x000055555555535d in task_1_segfault () at summary.c:12
#1  0x00005555555557c3 in main (argc=2, argv=0x7fffffffdb68) at summary.c:121

(gdb) print null_ptr
$1 = (int *) 0x0
```

1.2. Lab 2

```
Core was generated by `/home/huyen/Downloads/Session 6/summary 2'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  worker_thread (arg=0x0) at summary.c:32
32            *shared_ptr = 999;
```

Thread 1 crashed at summary.c:32 inside the worker_thread function.

```
Thread 1 (Thread 0x7ffff73fe6c0 (LWP 36293)):
#0  worker_thread (arg=0x0) at summary.c:32
```

```
Thread 3 (Thread 0x7ffff7bff6c0 (LWP 36292)):
#0  0x00007ffff7cad89d in __GI___libc_malloc (bytes=4) at ./malloc/malloc.c:3318
#1  0x0000555555555380 in allocator_thread (arg=0x0) at summary.c:19
```

Root Cause: Race Condition — Thread 1 and Thread 3 accessed the shared_ptr shared variable simultaneously without Mutex protection.

```
Thread 2 (Thread 0x7ffff7fa9740 (LWP 36289)):
#0  ... __futex_abstimed_wait_common64 ...
#3  ... in __pthread_clockjoin_ex ...
#4  in task_2_race_condition () at summary.c:42
#5  in main ... at summary.c:124
```

Command	Shortcut	Description
`break <function_name>`	`b`	Set a breakpoint at a specific function or code line
`run`	`r`	Start executing the program to be debugged
`next`	`n`	Execute the next line, stepping over function calls

Command	Shortcut	Description
`step`	`s`	Execute the next line, stepping into function calls if any
`backtrace`	`bt`	View the call stack (list of active stack frames)
`list`	`l`	Display source code lines around the current execution point
`print <expression>`	`p`	Print a variable value, call a function, or overwrite a variable value directly in RAM
`continue`	`c`	Resume program execution until the next breakpoint or program termination
`quit`	`q`	Exit GDB

1.3. How GDB Works ?

GDB directly intervenes in machine code and CPU hardware.

Breakpoint Mechanism

When typing the command break 20:

- GDB looks up the memory address corresponding to line 20.
- GDB uses the ptrace system call to read the first byte at that memory location.
- GDB overwrites that memory location with the byte 0xCC (0xCC is the breakpoint instruction named INT 3 - Interrupt 3).

When typing continue: GDB restores the original byte (removes 0xCC) and lets the CPU execute the instruction.

Watchpoint Mechanism

- GDB relies on Intel/AMD CPU hardware support.
- On the CPU, there are 4 special registers: Debug Registers (DR0, DR1, DR2, DR3).
- When typing the command watch x, GDB loads the memory address of variable x into the CPU's DR0 register and enables the mode: "If any instruction writes to this memory location, the CPU automatically triggers a hardware trap/interrupt."
- Because it is monitored directly by hardware circuits, the program continues to run at normal speed.

2. strace

2.1. Lab 3

Hangs

```
[pid 37780] <... clock_nanosleep resumed>0x71ada03fee70) = 0
[pid 37781] <... clock_nanosleep resumed>0x71ad9fbfde70) = 0
[pid 37780] futex(0x5c556e3030a0, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
[pid 37781] futex(0x5c556e303060, FUTEX_WAIT_PRIVATE, 2, NULL
```

Blocking Syscall: Both child threads (PID 37780 and PID 37781) are executing the futex system call with the FUTEX_WAIT_PRIVATE flag, showing an <unfinished ...> state.

Root Cause: The FUTEX_WAIT state indicates that the thread is sleeping while waiting for a Mutex lock to be released. Because both threads are stuck in a mutual lock contention (Deadlock), neither releases its lock, causing both to hang indefinitely at the futex syscall."

Fails silently

Running the syscall filtering command

```
strace -f -e trace=openat,clone,wait4,write ./summary 3
```

```
openat(AT_FDCWD, "/root/secret_admin_file.txt", O_RDONLY) = -1 EACCES (Permission denied)
write(3, "fopen failed: Permission denied\n", 32fopen failed: Permission denied
) = 32
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace: Process
37985 attached
, child_tidptr=0x77fdb5501a10) = 37985
[pid 37984] wait4(-1,  <unfinished ...>
[pid 37985] write(1, "[Child PID: 37985]\n", 19[Child PID: 37985]
) = 19
[pid 37985] +++ exited with 0 +++
<... wait4 resumed>NULL, 0, NULL)       = 37985
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=37985, si_uid=1000, si_status=0,
si_utime=0, si_stime=0} ---
+++ exited with 0 +++
```

The file open operation failed, returning -1 with the EACCES error code because the process is running under a non-root user and lacks access permissions to the root directory

strace acts as a middleman to intercept all system calls between the application and the kernel.

Differences between C library functions and actual system calls.

C Code Written in Program	Actual System Call Intercepted by `strace`	Explanation
`wait(NULL);`	`wait4(-1, ...)`	`wait()`, `waitpid()`, and `wait3()` in C are essentially wrappers that call the kernel's `wait4` system call.
`pid = fork();`	`clone(...)`	The `fork()` function actually invokes the `clone` syscall with flags to create a new process.
`fopen(...)`	`openat(...)`	The C standard library calls `openat` to open files safely.
`sleep(1)`	`clock_nanosleep(...)`	`sleep()` utilizes the kernel's high-resolution timer (`clock_nanosleep`) to pause the process.

Option	Description / Functionality
`-f`	For `fork()` / multithreading: Follow and log child processes as well.
`-ff -o log`	Separate output logs for each child process into individual files (`log.PID`).
`-e trace=...`	Filter: Display only specific syscalls of interest.
`-c`	Count total calls, total time, and errors for each syscall.
`-p <PID>`	Attach to a running system process.

2.2. How strace Works?

ptrace System Call

The ptrace system call allows one process to have full control and read the CPU registers of another process.

When running strace ./program:

- strace spawns a child process.
- The child process declares to the Kernel: "Allow strace to monitor me" (ptrace(PTRACE_TRACEME)).
- Afterward, the child process loads and executes ./summary.

Syscall-Enter and Syscall-Exit

Whenever the program requires OS assistance (opening a file, printing to the screen, creating a child process...), the CPU switches from User Mode to Kernel Mode.

Interception 1: When the program is about to invoke a system call (Syscall-Enter)

- Before the Kernel executes the instruction, it detects the ptrace flag -> freezes the program.
- The Kernel notifies strace, and strace reads the CPU registers (such as RDI, RSI, RDX...) to determine which function is being called (e.g., openat) and what parameters are passed (e.g., "/secret.txt").
- strace prints the first half of the log line: openat(AT_FDCWD, "/secret.txt", O_RDONLY) and lets the Kernel continue execution.

Interception 2: When the Kernel finishes executing the system call (Syscall-Exit)

- After detecting that the file does not exist, the Kernel writes the error code -1 into the RAX register.
- Before returning control back to the program, the Kernel freezes the program once again.
- strace reads the RAX register to check the return value.
- strace prints the remaining part: = -1 ENOENT (No such file or directory).
- The program then continues running.

-> 2–4x slower than normal execution

3. Valgrind

3.1. Lab 4

Run Valgrind Memcheck

```
valgrind --leak-check=full --track-origins=yes ./summary 4
```

```
==6026== Invalid read of size 4
==6026==    at 0x10969B: task_4_valgrind (summary.c:103)
==6026==    by 0x109819: main (summary.c:137)
==6026==  Address 0x4a7f4f0 is 0 bytes inside a block of size 4 free'd
==6026==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)

==6026== Invalid write of size 4
==6026==    at 0x1096B7: task_4_valgrind (summary.c:104)
==6026==    by 0x109819: main (summary.c:137)
==6026==  Address 0x4a7f4f0 is 0 bytes inside a block of size 4 free'd
==6026==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
```

-> Use-After-Free (Invalid read & Invalid write)

```
==6026== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==6026==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==6026==    by 0x109648: task_4_valgrind (summary.c:97)
==6026==    by 0x109819: main (summary.c:137)
==6026==
==6026== LEAK SUMMARY:
==6026==    definitely lost: 40 bytes in 1 blocks
```

-> Memory Leak (Definitely lost)

Fix

Fix Memory Leak: Add the free(leak_arr); statement after finishing with the array.

Fix Use-After-Free: Move the read and write operations BEFORE calling free(uaf_ptr);

```
==7043== HEAP SUMMARY:
==7043==     in use at exit: 0 bytes in 0 blocks
==7043==   total heap usage: 3 allocs, 3 frees, 1,068 bytes allocated
==7043==
==7043== All heap blocks were freed -- no leaks are possible
==7043==
==7043== For lists of detected and suppressed errors, rerun with: -s
==7043== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

-> 3 allocs, 3 frees

-> 0 errors

3.2. Management of V-bits (Valid-value bits) and Principles

Synthetic CPU and the V-bit Shadow

Valgrind constructs a synthetic CPU that runs alongside the real CPU.

Every data bit in RAM and all CPU registers has an associated V-bit:

- V = 1: The bit contains a valid (initialized) value.
- V = 0: The bit contains garbage (uninitialized) data.

When the CPU loads 4 bytes (32 bits) from RAM into a register, Valgrind concurrently loads the corresponding 32 V-bits along with them.

Principles

Case 1: Copying garbage data only -> Valgrind reports no errors

```
int a[10], b[10];
for (i = 0; i < 10; i++)
{
    j = a[i]; /* Assign garbage data from array a[10] to j */
    b[i] = j;  /* Assign garbage data from j to b[i] */
}
```

Valgrind remains completely silent. Because garbage data is merely being moved from one place to another, it has not yet caused any impact that alters the program's behavior.

Case 2: Using garbage data to make a decision -> Valgrind reports an error

```
for (i = 0; i < 10; i++)
{
    j += a[i]; /* Accumulating garbage data -> Valgrind dóe not report an error*/
}
if (j == 77) { /* Valgrind reports an error*/
    printf("hello there\n");
}
```

When perform the addition j += a[i], Valgrind quietly computes the V-bits of the result (resulting in j having V = 0). It does not report an error yet.

However, the moment you write if (j == 77):At this point, the program must make a decision (branching) on whether to print "hello there" to the screen for the user to see.

Whether it prints or not depends on a random garbage value -> Valgrind immediately flags an error

```
Conditional jump or move depends on uninitialised value(s)
```

Valgrind only checks V-bits at 3 points

- Using a garbage value as a memory address
- Using a garbage value to control execution flow
- Passing a garbage value into a System Call

Why doesn't Valgrind report an error immediately when a garbage variable is loaded into the CPU, but instead waits until it is actually used?

```
struct S {
    int x;   /* 4 bytes */
```

```
    char c;  /* 1 byte */
};
struct S s1, s2;
s1.x = 42;
s1.c = 'z';
s2 = s1; /* Struct assignment */
```

The actual size of struct S is 8 bytes (containing 3 padding bytes with garbage values).

If Valgrind reported an error the moment a garbage byte was read, almost every C program using structs would trigger a false positive in Valgrind due to memory padding mechanism

3.3. Valid-address (A) bits

What is the difference between A-bit and V-bit?

Criteria	A-bit (Valid-Address)	V-bit (Valid-Value)
Scope	**Only located in RAM memory** (CPU registers do not have addresses, so there is no A-bit).	Located in **both RAM and CPU registers**.
Unit	**1 A-bit for each Byte**.	**1 V-bit for each Bit** (8 V-bits / Byte).
If violated	Reports error `Invalid read` or `Invalid write`.	Reports error `conditional jump depends on uninitialised value`.

Rule: Whenever the program performs a Read or Write operation into RAM, Valgrind checks the A-bit first:

- A=1: Valid → Allows read/write.
- A=0: Forbidden memory location → Reports an error

Global Variables

As soon as the program starts up, the entire memory region containing global variables and string constants is automatically marked by Valgrind: A = 1.

Heap Memory Area

When calling malloc(20):

- Valgrind sets A=1 for exactly those 20 allocated bytes.
- Surrounding bytes (the Redzone area) all have A=0.
- Writing to a byte with A=0→ Reports an Invalid write error.

When calling free(ptr):

- Valgrind changes those 20 bytes back to A=0.

- Reading from that memory address afterwards (A=0) → Reports an Invalid read error (Use-After-Free)."

Revision #3
Created 2026-09-21 07:06:43 UTC by Huyen Nguyen
Updated 2026-09-23 04:22:02 UTC by Huyen Nguyen